



Safety and Security Critical Systems

Programming and Development of Embedded Systems

Safety Critical Systems

❖ Safety Critical Systems

- Systems whose failure or malfunction may result in one (or more) of the following
 - Loss of life or serious injury to people
 - Damage to properties/equipments
 - Damage to the environment
- Examples: Robotic surgery machines, braking system in vehicles and etc.



❖ Safety-critical systems are increasingly computer-based

❖ Safety-critical softwares fall into two classes:

- **Primary** safety-critical software: A software that is embedded as a controller in a system.
- **Secondary** safety-critical software: A software which indirectly is safety critical
 - E.g. A patient management software. A failure can cause an improper treatment for patients.

Safety Critical Systems

- ❖ **Hazard:** A condition which can cause harm or damage
- ❖ **Risk:** Probability that a system causes harm or damage in the case of hazardous event
- ❖ **Functional Safety** is absence of **unreasonable** risk of harming people or damaging the environment due to hazards caused by malfunctioning behaviour of systems
- ❖ **Dependability** is a general term
 - **Reliability:** Continuity of correct service.
 - **Availability:** Readiness for correct service.
 - **Safety:** Absence of harming people and damaging the environment.
- ❖ **Safety vs. Security** (or cyber security)
 - **Safety:** Absence of internal failures which can cause harm or damage
 - **Security:** Protection against external threats which can cause harm or damage
 - You can have Security without Safety. But, in most cases, not Safety without Security.

Safety Critical Systems

❖ Types of Safety Critical System in Term of Reliability

- **Fail-operational** systems: Remaining part of the system can complete the operation
 - E.g. a remote keyless system
- **Fail-safe** systems: They switch to a safe mode and usually inform an operator
 - E.g. Windows, Insulin pumps and etc.
- **Fail-secure** systems: They become secure usually by locking up to minimize harm
 - E.g. Electronic doors, lock during power failures
- **Fail-passive** systems: They become passive and hand over control to an operator;
 - E.g. An aircraft autopilot
- **Fault-tolerant** systems: They continue to operate in the case of failure
 - Usually by detecting the at risk components and replace them before they result in any risk
 - E.g. cooling system in a nuclear reactor



Safety Critical Systems

- ❖ Risk can never be completely eliminated. Probabilities can be reduced.
- ❖ Development of Safety Critical Systems
 - Creating systems that always operate safely by minimizing, controlling and reducing hazards
 - Is performed according to the **state of the art** in all aspects of safety.
 - Reducing risks as far as "reasonably practicable" - good enough.
 - The burden of proof is with the manufacturer.
 - Documented proof must be produced.
 - International standards are required.
- ❖ There are different standards in different industries.
 - IEC 61508 for industry in general
 - IEC 62304 for medical industry
 - ISO 26262 for road vehicle industry

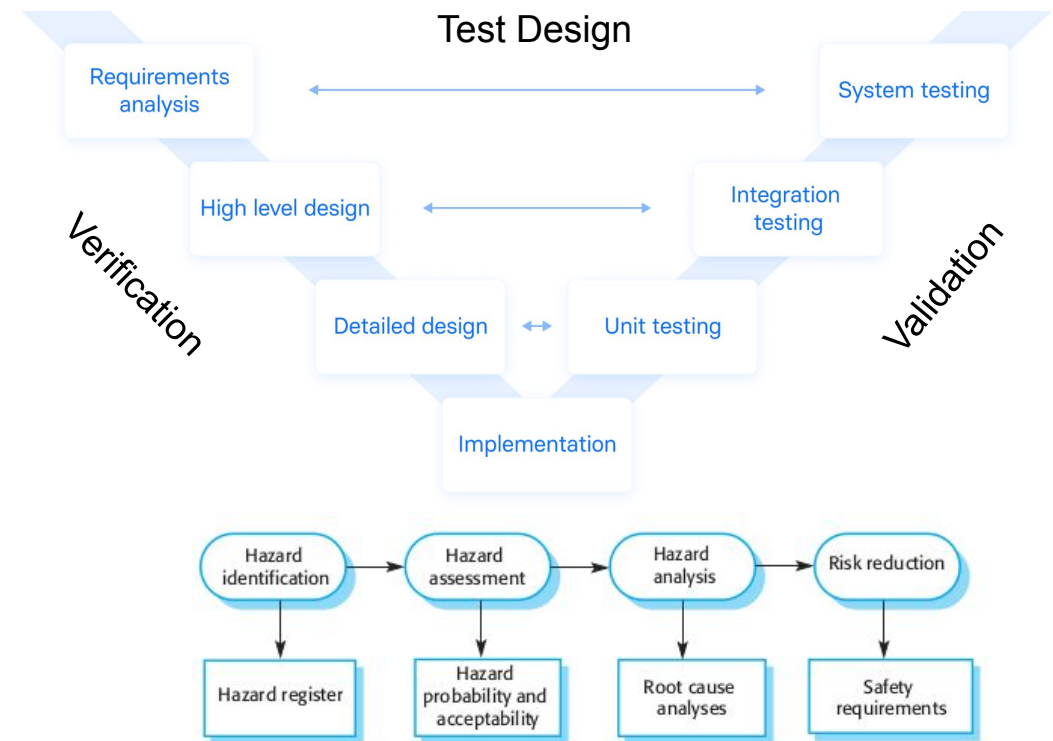
Automotive Functional Safety - ISO 26262

❖ ISO 26262: Functional Safety for Road Vehicles

- Part 1: Vocabulary
- Part 2: Management of functional safety
- Part 3: Concept phase
- Part 4: Product development at the system level
- Part 5: Product development at the hardware level
- ***Part 6: Product development at the software level***
- Part 7: Production and operation
- Part 8: Supporting processes
- Part 9: Automotive Safety Integrity Level (ASIL)-oriented and safety-oriented analyses
- ***Part 10: Guideline on ISO 26262***

ISO 26262 Product development

- ❖ ISO 26262 uses a classic V-Model framework at **system, hardware and software** levels
- ❖ ISO 26262 Part 6: Product development at the software level
 - Safety requirements Specification.
 - Architectural design.
 - Unit design and implementation.
 - Unit testing.
 - Integration testing
 - System testing
 - Verification and validation of safety requirements in each phase.



Safety Critical Software Systems (ISO 26262 Guidelines)

- ❖ Modelling and coding guidelines
 - Enforcement of low complexity
 - Use of language subsets
 - Like MISRA C/C++ and AUTOSAR C++
 - Enforcement of strong typing
 - Use of defensive implementation techniques
 - Use of well-trusted design principles
 - Use of unambiguous graphical representation
 - Use of naming conventions
 - Use of style guides
 - Like Visual Studio Style Guides for C/C++

Safety Critical Software Systems (ISO 26262 Guidelines)

- ❖ Principles for software architectural design
 - Hierarchical structure of software components
 - Restricted size and complexity of software components
 - Restricted size of interfaces
 - Strong cohesion within each software component
 - Loose coupling between software components
 - Appropriate scheduling properties (timing property of the components)
 - Appropriate management of shared resources
 - Appropriate isolation of the software components
 - Restricted use of interrupts
 - Use of design notation, like UML, flow charts and structure charts

Safety Critical Software Systems (ISO 26262 Guidelines)

- ❖ Design principles for software unit design and implementation
 - Use of semi-formal notation; like UML, timing diagram and etc.
 - One entry and one exit point in subprograms and functions
 - No dynamic objects or variables, or else online test during their creation
 - Initialization of variables
 - No multiple use of variable names
 - Avoid global variables or else justify their usage
 - Restricted use of pointers
 - No implicit type conversions
 - No hidden data flow or control flow
 - No unconditional jumps
 - No recursions

Safety Critical Software Systems (ISO 26262 Guidelines)

❖ Methods for software verification

- Walk-through
- Pair-programming
- Semi-formal verification
- Control flow and data flow analysis
- Static code analysis; like [Coverity](#), [Helix QAC](#), [IAR](#), [CodeSonar](#), [Parasoft C/C++test](#) and etc.
- Requirement-based test
- Interface test
- Fault injection test
- Resource usage evaluation
- Analysis of boundary values

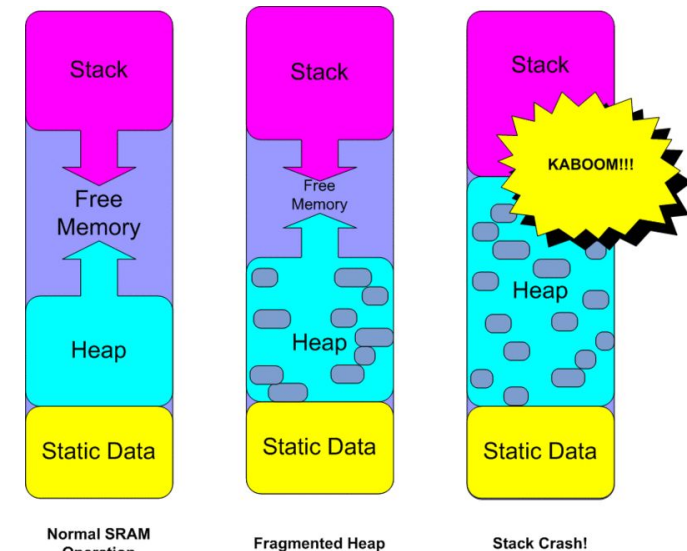
Safety Critical Software Systems (ISO 26262 Guidelines)

❖ Structural coverage metrics

- Statement coverage
- Branch coverage
- Modified condition/decision coverage (MC/DC)
 - Each entry and exit point are invoked
 - Each decision takes every possible outcome
 - Each condition in a decision takes every possible outcome
 - Each condition in a decision is shown to independently affect the outcome of the decision.
 - Independence of a condition is shown by proving that only one condition changes at a time.
- Function coverage
- Call coverage

Embedded Programming Main Concerns

- ❖ The main purpose of dynamic memory allocation & deallocation
 - Multiple processes can dynamically share all the available RAM memory
 - E.g. a multi-process system with a large and variable or unknown amount of memory
- ❖ In addition to memory leak, there are some problems with dynamic memory allocation/deallocation
 - It might fail
 - Memory fragmentation
 - It is slow; memory needs to be scanned to find free space
 - Memory management is required
 - Its execution time is non-deterministic and varying
 - In the case of time-critical systems it is unacceptable



Embedded Programming Main Concerns

- ❖ Dynamic memory allocation is not safe, secure and deterministic.
- ❖ Generally dynamic memory allocation/deallocation is not used in safety critical systems:
 - It can be harmful and it is hard to handle failures
 - Safety critical systems are deterministic (no dynamic behaviors)
 - You always know the required amount of memory for the worst case
- ❖ Alternatives for dynamic memory allocation/deallocation
 - Static memory allocation
 - Static memory preallocation and a safe memory management implementation
 - E.g. A memory pool or a stack-based memory management
 - Memory allocation at startup is generally safe; but not freeing
 - Creating single-instance modules which can manage a known number of sets of data by determining the number of required instances and statically allocating memory for them

Embedded Programming Main Concerns

- ❖ STL containers by default use dynamic memory allocation/deallocation either directly or indirectly.
- ❖ Run-time operations such as dynamic polymorphism, exceptions and etc.
- ❖ Certain features of programming languages(e.g. templates), if not employed correctly, could result in code bloat at machine code level.
 - In computer programming, [code bloat](#) is the production of program code (source code or machine code) that is perceived as unnecessarily long, slow, or otherwise wasteful of resources.
- ❖ C++ in embedded programming is used as a better C
- ❖ Not all features of C/C++ are safe and not everything in the standard libraries can be used
- ❖ Rich features in C++ are indeed resource-intensive, yet it is guaranteed that
 - There is zero-cost rule i.e. you do not pay for what you are not using.
 - Costs are compared to C, and basically are: Size of the code, Execution time and Memory usage

C++ As An Embedded Language

- ❖ In order to ensure *concerning* features of C++ are not employed and the language is used *properly*; different industrial consortia provide sets of rules, guidelines, tools, and standard on how to use C++.
- ❖ These “guidelines” may share the same views over many concepts, yet they have different regulations to meet specific requirements.
- ❖ Some examples are:
 - Embedded C++ (EC++)
 - Motor Industry Software Reliability Association (MISRA) C++
 - AUTomotive Open System ARchitecture (AUTOSAR) C++

C++ “concerning” Features
new/delete
<i>Templates</i>
Exceptions
Run-time type information
Pure Virtual Functions
Multiple Inheritances

Embedded C++ (EC++)

- ❖ Embedded C++ (EC++) is a dialect of the C++ programming language for embedded systems. It was defined by an industry group led by major Japanese central processing unit (CPU) manufacturers [...] to address the shortcomings of C++ for embedded applications [...] a restricted subset of C++ (based on Embedded C++) has been adopted by Apple Inc. as the exclusive programming language to create all I/O Kit device drivers for Apple's macOS, iPadOS and iOS operating systems of the popular Macintosh, iPhone, and iPad products.

Feature	ISO/ANSI C C++	Embedded C++
Exception handling	✓	✗
Multiple inheritance	✓	✗
<code>mutable</code> , a storage class specifier	✓	✗
Namespaces	✓	✗
Templates	✓	✗
Run-time type information (<code>typeid</code>)	✓	✗
Style casts (<code>static_cast</code> , <code>dynamic_cast</code> , <code>reinterpret_cast</code> , and <code>const_cast</code>)	✓	✗
Virtual base classes	✓	✗

MISRA C++

- ❖ Motor Industry Software Reliability Association ([MISRA](https://www.misra-cc.org/)), provides a set of software development guidelines to facilitate code safety, security, portability, and reliability in the context of embedded system.
- ❖ Initiated for the C language, and later issued guidelines for C++; the guidelines, since introduction, were heavily adopted by industries dealing with critical systems.
- ❖ There are three standard released of MISRA C++ at 2008, 2012, and 2023 which are referenced to by other guidelines, like AUTOSAR C++ 14.
- ❖ MISRA guidelines focus on software engineering, as well as programming language and coding. It also defines steps necessary to make sure an implementation is compliant to the subset.



MISRA C++

❖ MISRA guidelines are categorized into three classes:



➤ **Required** rules: These are **mandatory** requirements

- C++ code claimed to conform to MISRA C++ shall comply with every “Required” rule.
- In the case of deviation, a formal deviation is required

➤ **Advisory** rules: These are requirements and should normally be followed; but they do not have the mandatory status of “Required” rules. .

- Note that the status of “Advisory” does not mean that these items can be ignored, but that they should be followed as far as they are reasonably practical.

➤ **Document** rules: These are mandatory requirements placed on the developer whenever the associated feature is used within code.

❖ There are tools for both static and dynamic analysis to check whether a given source code complies with MISRA. In some industries MISRA compliance is a requirement.

MISRA C++: Some Required Rules



- ❖ A project shall not contain unreachable code.
- ❖ A project shall not contain unused variables.
- ❖ A project shall not contain unused type declarations.
- ❖ The value returned by a function having a non-void return type that is not an overloaded operator shall always be used.
- ❖ If a function generates error information, then that error information shall be tested.
- ❖ Every defined function shall be called at least once.
- ❖ Literal suffixes shall be upper case.
- ❖ When an array is declared, its size shall either be stated explicitly or defined implicitly by initialization.
- ❖ An identifier declared to be an object or type shall be defined in a block that minimizes its visibility.
- ❖ Value of an expression shall be the same under any order of evaluation that the standard permits.
- ❖ Unions shall not be used.
- ❖ All variables shall have a defined value before they are used.

MISRA C++: Some Advisory Rules



- ❖ Sections of code should not be “commented out” using C++ comments.
- ❖ The identifier name of a non-member object or function with static storage duration should not be reused.
- ❖ typedefs indicate size and signedness should be used in place of the basic numerical types.
- ❖ Casts from a base class to a derived class should not be performed on polymorphic types.
- ❖ A cast should not convert a pointer type to an integral type.
- ❖ The increment (++) and decrement (--) operators should not be mixed with other operators in an expression.
- ❖ Functions should not call themselves, either directly or indirectly.
- ❖ An exception object should not have pointer type.
- ❖ There should be at least one exception handler to catch all otherwise unhandled exceptions

MISRA C++: Some Document Rules



- ❖ Minimization of run-time failures shall be ensured by the use of at least one of:
 - static analysis tools/techniques;
 - dynamic analysis tools/techniques;
 - explicit coding of checks to handle run-time faults.
- ❖ Floating-point implementations shall comply with a defined floating-point standard.
- ❖ The implementation of integer division in the chosen compiler shall be determined and documented.
- ❖ The character set and the corresponding encoding shall be documented.
- ❖ All usage of assembler shall be documented.
- ❖ Exceptions shall only be used for error handling.
- ❖ All library codes shall conform to MISRA C++.

in October 2023, MISRA integrated AUTOSAR C++ 14 and published one standard for C++17 in a joint effort.

- ❖ AUTomotive Open System ARchitecture (AUTOSAR) aims to create and establish an open and standardized software architecture for automotive ECUs to ensure scalability and transferability of software as well as focusing on availability and safety requirements.
- ❖ AUTOSAR specifies coding guidelines for C++14, in safety-related and critical systems. MISRA C++ 2008 is a prerequisite for AUTOSAR C++14 – most guidelines have been adopted from MISRA “as-is”, while there are some which have been modified or dismissed.
- ❖ AUTOSAR C++14 rules are mostly enforceable by static analysis. However, for some of the rules there need to be a manual review.
- ❖ AUTOSAR C++14 could be considered as a more relaxed standard compared to the one MISRA provides. For instance, dynamic memory usage is allowed in AUTOSAR while banned in MISRA C++.

AUTOSAR C++ Guidelines



- AUTOSAR provides guidelines for some of the concerning features of C++ to ensure they are used in the correct (less risky) manners. However, there are some features which their usage is banned.
- AUTOSAR provides tools and instructions for both static and dynamic analysis of the software to verify its level of compliance with the guidelines. Simplicity of the code is something which should be measured in AUTOSAR as well.

C++ features which shall not be used

malloc and free
C-style casts
const_cast
dynamic_cast
reinterpret_cast
goto statement
Return type deduction
typedef specifier
asm declaration
Variadic arguments
Unions
Multiple inheritance
friend declaration
User-defined literals
Function-try-blocks
Dynamic exception specification
#pragma directives

C++ features which could be used under AUTOSAR C++ 14 guidelines

Dynamic memory management	List initialization
Floating-point arithmetic	Bit-fields
Operators new and delete	Inheritance
Sized deallocation	Virtual Functions
Namespaces	override specifier
Fixed width integer types	final specifier
nullptr pointer literal	Defaulted and deleted functions
static_cast	Delegation constructors
Lambda expressions	Member initializer list
Generic lambda expression	Non-static member initializer
Binary literals	explicit specifier
Range-based for loops	Move semantics
constexpr specifier	Digit sequence separators
auto specifier	Variadic templates
decltype specifier	Variable templates
Trailing return type syntax	Exceptions
using specifier	noexcept specifier
Scoped enumerations	Static assertion
std::initializer_list	
Default arguments	

Side Effects and Sequence Points

- ❖ The evaluation order of operands is undefined for most of the operators in C++
 - Therefore some unexpected results may occur when using certain operators
 - A program shall not use undefined behaviours in the language
 - These unexpected results are caused by **side effects**
- ❖ Any operation that affects storage of an operand has a **side effect**
- ❖ Any operation has a **persistent side effect** if it does anything beyond its life
- ❖ Conflicting **side effects** are a kind of undefined behavior.
- ❖ C++ guarantees that all side effects on an expression will be concluded by the next **sequence point** in a program.

```
int x = 20;
int f1(void) { x += 10; return x; }
int f2(void) { x -= 5; return x; }
int main(void) {
    // 30 + 25 or 25 + 15
    int p = f1() + f2();
    printf("p = %d", p);
    return 0;
}
```

```
int x = 3; printf("%d, %d, %d, %d\n", x++, x, ++x, x);
```

Side Effects and Sequence Points

- ❖ **Sequence points** are checkpoints in a program at which the compiler ensures that operations in an expression are concluded.
- ❖ The most known sequence point is the **semicolon** marking the end of a statement.
- ❖ Some other sequence points are as follows:
 - `expr1, expr2` (the comma operator)
 - `expr1 && expr2` (the logical AND operator)
 - `expr1 || expr2` (the logical OR operator)
 - `expr1 ? expr2 : expr3` (the conditional operator)
- ❖ Using the evaluation order of side effects, when none is guaranteed, is risky
- ❖ Best practice: no more than one side effect on an object in an expression.

Analysis

- ❖ Analysis to find bugs and enhance the code quality!
 - Static analysis tools are used to analyse a program source code
 - Dynamic analysis tool are used to analyse a program executable binary
- ❖ Some free analysis tools:
 - Static: [Cppcheck](#), [Clang Analyzer](#), [CodeChecker](#), [Cpplint](#)
 - Dynamic: [Valgrind](#), [GNU gprof](#), [uftrace](#), [Orbit](#), [Clang Sanitizers](#)
- ❖ G++ can also generate warnings on a lot of questionable and unsafe features of C++. It is also possible to convert warnings to errors.
 - [Options That Control Static Analysis](#)
 - [Options to Request or Suppress Warnings](#)
 - [Options Controlling C++ Dialect](#)

```
-pedantic -Wall -Wextra -Wcast-align -Wcast-qual -Wctor-dtor-privacy -Wdisabled-  
optimization -Wformat=2 -Winit-self -Wlogical-op -Wmissing-declarations -Wmissing-  
include-dirs -Wnoexcept -Wold-style-cast -Woverloaded-virtual -Wredundant-decls -  
Wshadow -Wsign-conversion -Wsign-promo -Wstrict-null-sentinel -Wstrict-overflow=5 -  
Wswitch-default -Wundef -Werror -Wno-unused
```

Safety Critical Systems

❖ In an embedded software, both safety and security are important.

➤ Safety - Ability of a system to function

■ Without causing harm to objects or people with a sufficient level of reliability

➤ Security - Ability of a system to protect

 [Keeping Embedded Secure](#)

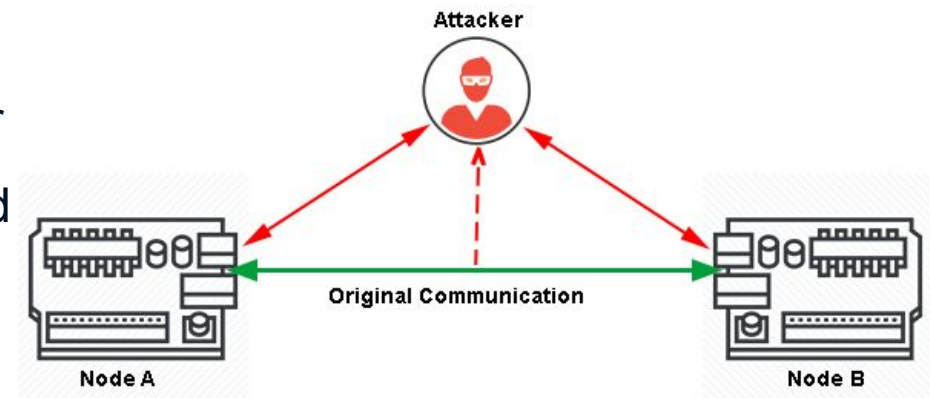
■ Against external attacks e.g. hackers and leakage of sensitive data

❖ Security: It is all about **C.I.A**

➤ **Confidentiality** - allows the sender and receiver to share information only in the way they intend

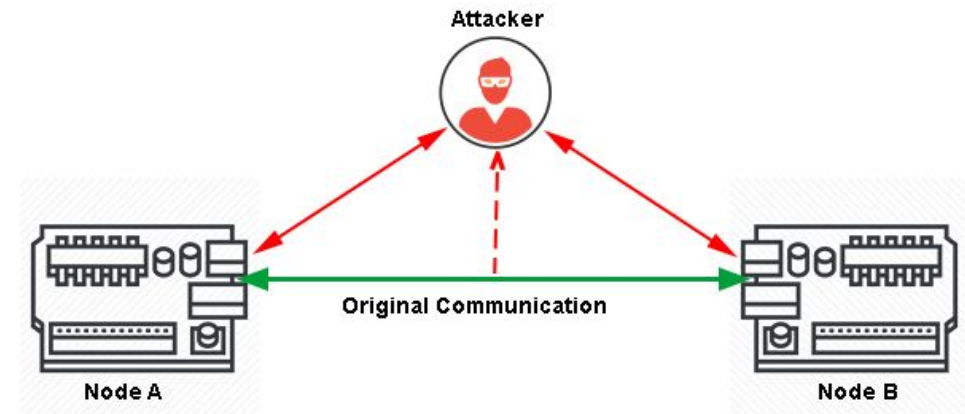
➤ **Integrity** – allows the receiver to verify that the data has not been altered during transmission

➤ **Authentication** - allows the receiver of information to be certain where it came from



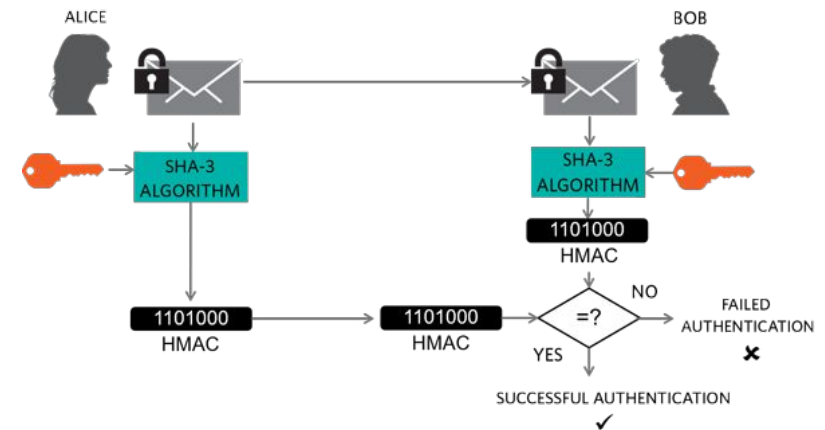
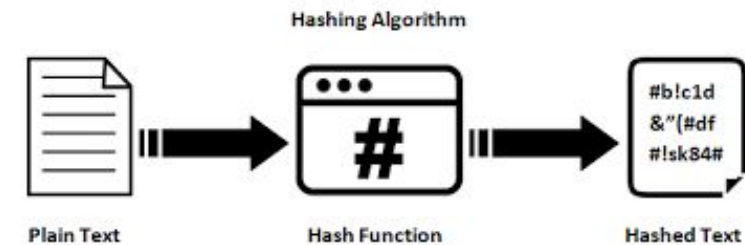
Security - Data Integrity

- ❖ Note that the attacker can listen to the communication and can send and receive data
- ❖ If the communication is not secure
 - Sensitive information is leaked
 - The attacker can act as man in the middle
 - Pretends it is the sender and receiver
- ❖ Data Integrity
 - A checksum be used to check if a message has been altered during transmission or not
 - There are [different algorithms](#) to checksum a message. E.g. CRC, MD, SHA and etc.
 - [A hash function](#) is a one-way function which takes a message of any size and generates a unique and fixed size hash value from the message.
 - There is no way to reverse the process and convert the hash value to the original message



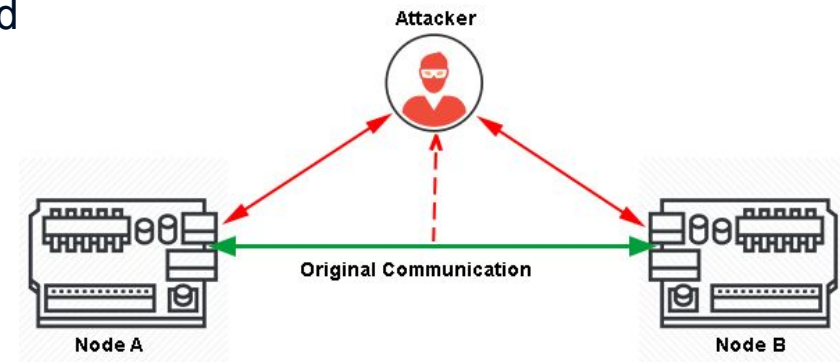
Security - Data Integrity

- ❖ **Hash** algorithms are used in data integrity and creating digital signatures
- ❖ SHA ([Secure Hash Algorithm](#)) are cryptographic hash functions used to verify data integrity.
 - There are SHA1, SHA2 and SHA3 types
 - E.g. SHA1 generates a 160-bit hash value(20 bytes)
 - E.g. SHA256 is a SHA2 type which takes a message with any length and generates a 256-bit hash value (32 bytes)
- ❖ **HMAC**(Hash-based message authentication code) is a cryptographic authentication technique that uses a hash function and a secret key.



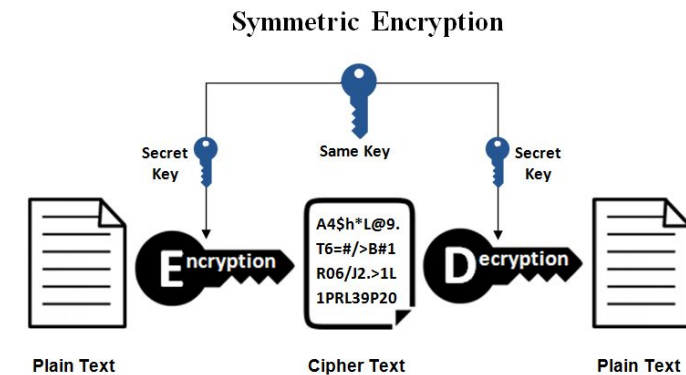
Security - Cryptography

- ❖ **Cryptography** is the practice and study of techniques for secure communication in the presence of third parties called adversaries.
 - **Key:** A secret like a password used to encrypt and decrypt information.
 - There are a few different types of keys used in cryptography.
 - **Encryption:** It is the process of locking up information using cryptography.
 - Data is turned into ciphers(encrypted data)
 - The encrypted data is not understandable a third
 - **Decryption:** The process of unlocking the encrypted information.



Security - Cryptography

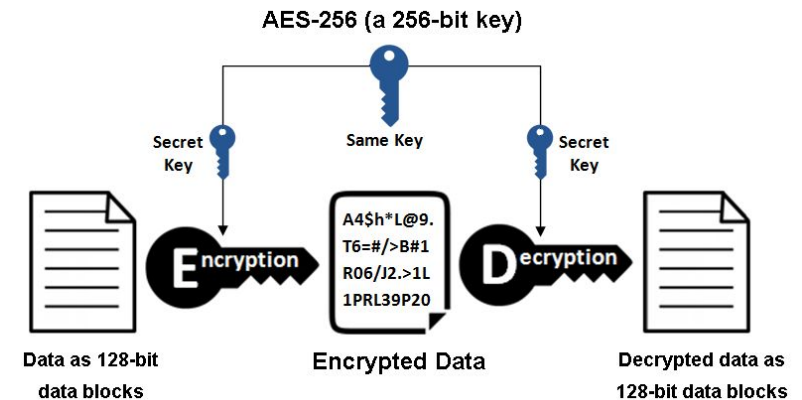
- ❖ There are **Symmetric** and **Asymmetric** encryptions
- ❖ Symmetric encryption
 - In this method only one shared secret key is used by the parties to encrypt and decrypt information
 - The process is very fast and it only provides confidentiality.
 - Algorithms like DES, **AES**([Advanced Encryption Standard](#)) and etc. are used
 - AES is a block cipher with a data block of 128 bits and three key lengths: 128, 192, or 256 bits.



Security - Cryptography

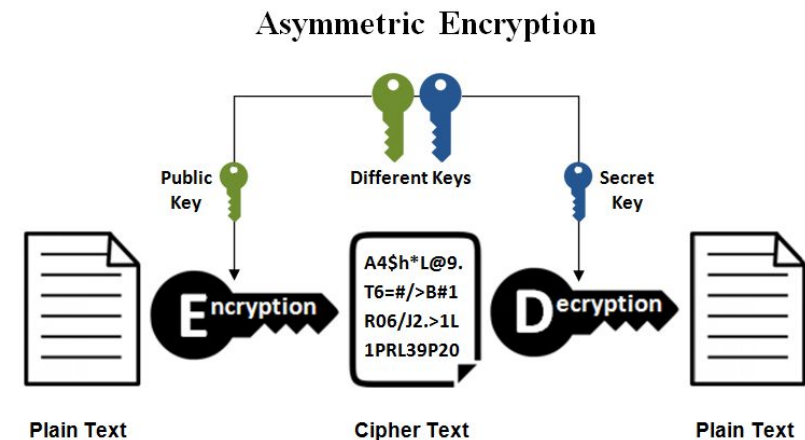
❖ AES-256 as [the strongest encryption standard](#) uses

- A 256-bit secret key to encrypt and decrypt blocks of 128-bit data. Means that the data shall be blocks of 128 bits and for each block the encrypted data is 128 bits. Padding is used.



❖ Asymmetric encryption

- In this method a pair of keys (public and private) is used by the parties to encrypt and decrypt data
- It is also called public key encryption. One of the keys is used to encrypt and the other one is used to decrypt information. The public key is shared.



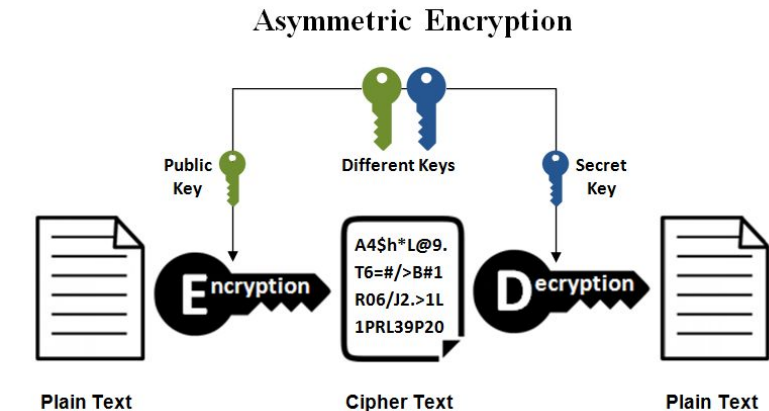
Security - Cryptography

❖ Asymmetric encryption ...

- The process is slow. It provides both confidentiality and authenticity. Usually it is used to sign and share secrets like AES keys. The private key is used to sign secrets (encrypt with the private key and decrypt with the public key)
- Algorithms like ECC, DSA and **RSA** are used.

❖ **RSA** (Rivest–Shamir–Adleman) algorithm

- A pair of big keys is used; usually 2048-bit keys
- The data block size and the cipher size are equal and their sizes are equal to the size of keys
- Padding is used to pad data block



🖨 [Asymmetric Encryption - Simply explained](#)

🖨 [Symmetric Key and Public Key Encryption](#)

🖨 [RSA algorithm](#)

🖨 [Symmetric vs. Asymmetric Key Encryption](#)

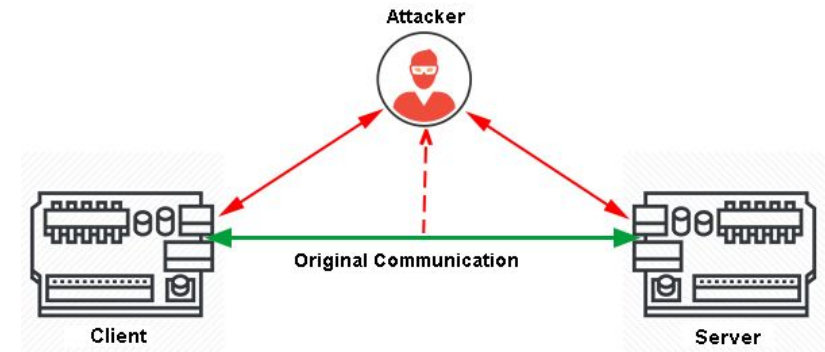
🖨 [Symmetric vs. Asymmetric Encryption](#)

🖨 [Public Key Cryptography](#)

🖨 [RSA Key Generator](#)

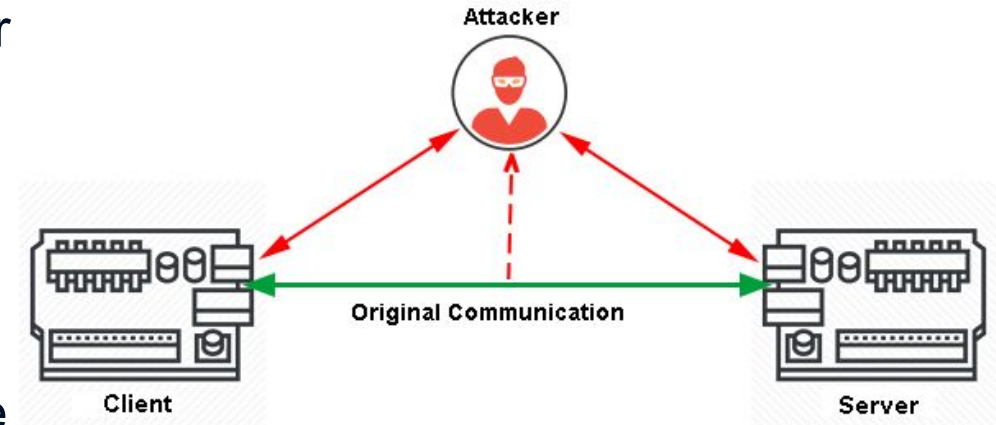
Security - Example

- ❖ Suppose that a server has a service restricted to clients that have secret IDs shared with the server. It means that the server has a list of client keys and it gives the service to a client whose secret ID has been registered in the server's list. If a client wants to get the service, first it should send its ID in a secure way to the server and the server shall check the client secret ID validity.
- ❖ Using HMAC, SHA256, AES-256, DH-2048 and RSA-2048 make the communication secure.
- ❖ To provide authentication and data integrity for sessions we use HMAC-SHA256
 - All the transactions shall be protected using HMAC-SHA256
 - The sender calculates the HMAC of its message and append it to the message
 - The receiver recalculates the HMAC for the received message and compare it with the received one. If they are equal, it means that the message is authenticated and it has not been altered.



Security - Example

- ❖ If the client already does not have the server public key and wants to get it in a secure way, we face with a problem. Because the attacker can pretend to be the server and send its public key to the client. In this case



the client does not know it has received the public key of the server or the public key of the man in the middle. To solve this problem a trusted third-party shall approve identity of the server using a [certificate](#).

➤ E.g. Using a [DigiCert TLS/SSL Certificate](#)

- ❖ Assume that the client has the server's public key.

Safety and Security Critical Systems

❖ Some useful links

- [Safety Critical System](#)
- [Automotive functional safety](#)
- [Safe and Reliable Computer Control Systems](#)
- [Functional Safety with ISO 26262](#)
- [Autonomous Vehicle Standards](#)
- [AUTOSAR Guidelines to Use C++14](#)
- [A Quick Guide to ISO 26262](#)
- [HMAC explained](#)
- [Doxygen](#)
- [C++ for Embedded: Advantages, Disadvantages, and Myths](#)
- [C++ TUTORIAL - EMBEDDED SYSTEMS PROGRAMMING](#)
- [Modern C++ in embedded systems](#)
- [C++ On Embedded Systems](#)
- [Modern C++ in Embedded Systems](#)
- [C++ Patterns to Make Embedded Programming More Productive](#)
- [C++ in the World of Embedded Systems](#)